CloudOps Manager

Software Requirements Specification

cloud management platform

ghaya bedoui , nour keskes , meriem dellai , eya abid

Version 1.0

Stack Next.js, NestJS, PostgreSQL, Docker Compose

Scope Development phase (cloud deployment planned for later)

Contents

1 Context **3**

2 Project objectives **3**

3 System actors **4**

4 Actions associated with each actor **4**

5 Functional requirements **6**

5.1 Authentication and users 6

5.2 Projects 6

5.3 Servers 6

5.4 Applications 6

5.5 Monitoring 7

5.6 Alerts and incidents 7

5.7 Dashboard 7

5.8 Cross-cutting API behaviour 8

5.9 Frontend behaviour 8

5.10 Deployment 8

6 Non-functional requirements **8**

7 System architecture **10**

7.1 Data model 10

7.2 API design and conventions 11

7.3 Code organisation 12

7.4 Docker Compose setup 12

8 Technology choices **13**

9 Constraints, assumptions and delivery phases **13**

1 Context

CloudOps Manager is a web platform that lets a company manage its projects, servers and cloud applications from one place, watch their health, and react to incidents.

Business problem. The company runs many projects across several VPS, VM and cloud providers. Server specifications, application versions, resource usage and incidents are scattered across spreadsheets, provider consoles and chat messages. Nobody has one reliable view of what runs where, in which environment, and whether it is healthy.

Proposed solution. A full-stack application with a REST API (NestJS) and a web interface (Next.js) backed by PostgreSQL. It stores the inventory (projects, teams, servers, applications), records CPU and RAM measurements, raises alerts when configurable thresholds are crossed, and summarises everything on a dashboard. Access is controlled by role.

Delivery context.

- The application is delivered as containers and must run on a VM, VPS or cloud environment.
- Monitoring data may be simulated. Real collection is allowed only on a controlled test infrastructure.
- Authentication, role-based access and an auditable API are part of the first release, not later additions.

Scope of this document. This specification covers the development phase: what the system must do and how it is built with Next.js, NestJS and PostgreSQL. Cloud deployment (AWS or Azure) is planned for later and is treated as a deployment target, not as part of the core design.

2 Project objectives

The project delivers one deployable platform that centralises cloud resource management, monitoring and alerting, and that can be installed with a single command.

General objective. Design and build a secure, containerised full-stack application that manages projects, servers and applications, visualises monitoring indicators and handles alerts.

Specific objectives.

ID	Objective	How it is verified
O1	Centralise the inventory of projects, teams, servers and applications	CRUD screens and endpoints for each entity, with search, filters and pagination
O2	Secure access with JWT and three roles (Admin, DevOps, Viewer)	Guards reject unauthorised calls with 401 or 403
O3	Supervise servers through CPU/RAM metrics, history and availability	Charts refresh periodically from simulated or real data
O4	Detect incidents with configurable thresholds and keep their history	An alert is created when a metric crosses a threshold
O5	Give a global view on a dashboard	Counters, indicators and charts with filters
O6	Deliver a reproducible, containerised deployment	`docker compose up` starts the full stack with a persistent database
O7	Ensure quality and documentation	Jest tests, Swagger, README, architecture and data model documentation

Technical targets.

- A relational model of linked entities with enforced constraints.
- A REST API fully documented in Swagger.
- DTO and pipe validation, centralised exception handling, pagination, search and filters.
- Three roles with distinct permissions, enforced on the backend.
- Responsive interface with real API integration and loading and error states.
- Unit tests plus integration tests on key endpoints; a clean, reviewable Git history.

Planned extensions. CI/CD with GitHub Actions, real-time updates over WebSocket, real metrics collection, deployment on AWS or Azure.

3 System actors

Three human roles use the platform, and two automated actors feed it. Each user holds exactly one role.

Actor	Type	Description	Access level
Admin	Human, primary	Owns the platform: manages users, roles, and all resources and thresholds	Full
DevOps	Human, primary	Operates infrastructure: manages servers, applications, projects and alert rules, and handles incidents	Read and write on operational data, no user management
Viewer	Human, secondary	Stakeholder who only consults inventory, dashboard and incidents	Read only
Monitoring Simulator / Collector	System	Produces CPU/RAM measurements, either simulated or collected from a controlled test server	Writes metrics only
Scheduler	System	Runs periodic jobs: metric generation, threshold evaluation, availability checks	Internal, no user session

Notes.

- Anonymous visitors can only reach the login page (and registration if enabled).
- The first Admin is created by a seed script or a one-time setup endpoint, since open self-registration must not grant privileged roles.
- The simulator and scheduler are internal NestJS components, not external services, so they need no login.

4 Actions associated with each actor

Admin can do everything DevOps can, DevOps can do everything Viewer can, and each step up adds write power. The matrix below is the reference for Guards and UI visibility (C = create, R = read, U = update, D = delete).

Resource / action	Admin	DevOps	Viewer
Log in, view and edit own profile, log out	Yes	Yes	Yes

Resource / action	Admin	DevOps	Viewer
Manage users and roles (C R U D)	Yes	No	No
Projects, teams, assignments, environments	C R U D	C R U D	R
Servers (VPS/VM)	C R U D	C R U D	R
Applications	C R U D	C R U D	R
View metrics, history and availability	R	R	R
Alert rules and thresholds	C R U D	C R U D	R
Acknowledge and resolve incidents	Yes	Yes	No
View alert and incident history	R	R	R
Dashboard (view, filter)	Yes	Yes	Yes
Delete projects or servers that still have dependents	Yes	No	No

Admin

- Create, edit, deactivate and delete user accounts; assign or change roles.
- Perform all DevOps actions.
- Force-delete resources with dependents and review the audit trail of changes.

DevOps

- Create and maintain projects, environments, teams and assignments.
- Register servers, update their specs and status, and attach applications to them.
- Define alert rules (metric, threshold, severity) and enable or disable them.
- Acknowledge, comment on and resolve incidents.
- Trigger or pause metric simulation on test servers.

Viewer

- Browse projects, servers, applications and their relations, using search and filters.
- Read metric charts, alert history and the global dashboard.
- Edit only their own profile and password.

System actors

- Simulator/Collector: generates a CPU and RAM sample per server at a fixed interval and stores it.
- Scheduler: evaluates each new sample against active thresholds, opens an alert when a threshold is crossed, closes it when the value returns to normal, and marks servers unavailable when samples stop arriving.

5 Functional requirements

Requirements are grouped by module. Priority: **M** = must (part of the first release), **S** = should (added value, scheduled after the must items).

5.1 Authentication and users

ID	Requirement	Prio.
FR-A1	A user can log in with email and password and receives a signed JWT (access token, with an expiry)	M
FR-A2	Passwords are hashed (bcrypt or argon2) and never returned by the API	M
FR-A3	The first Admin is created by seed; Admins create further users and assign roles (Admin, DevOps, Viewer)	M
FR-A4	Optional public registration creates Viewer accounts only	S
FR-A5	A user can view and edit their profile (name, email, password)	M
FR-A6	Protected routes require a valid JWT; role Guards enforce the permission matrix	M
FR-A7	Refresh tokens and logout with token invalidation	S

5.2 Projects

ID	Requirement	Prio.
FR-P1	CRUD on projects (name, description, owner, status, dates)	M
FR-P2	Manage teams and assign users to projects with a project role	M
FR-P3	Define environments per project (dev, staging, production)	M
FR-P4	List projects with pagination, text search and filters (status, environment, team)	M

5.3 Servers

ID	Requirement	Prio.
FR-S1	CRUD on VPS/VM records: name, provider, region, CPU cores, RAM, storage, IP address, status	M
FR-S2	Link a server to a project and an environment	M
FR-S3	Validate input (IP format, positive numeric specs, allowed status values)	M
FR-S4	List servers with pagination, search and filters (provider, region, status, project)	M
FR-S5	Server detail page showing specs, hosted applications, latest metrics and open alerts	M

5.4 Applications

ID	Requirement	Prio.
FR-AP1	CRUD on applications: name, version, environment, status	M
FR-AP2	Attach an application to one server (and its project)	M
FR-AP3	Record version changes with date and author	S
FR-AP4	Filter applications by server, environment, status	M

5.5 Monitoring

ID	Requirement	Prio.
FR-M1	Store CPU and RAM usage (%) per server with a timestamp	M
FR-M2	A scheduled job generates simulated metrics at a fixed interval for every active server	M
FR-M3	Provide a metrics history endpoint with time-range filter (last hour, day, week)	M
FR-M4	Compute availability (uptime %) over a period from received samples	M
FR-M5	Show line charts of CPU and RAM and refresh them periodically (polling)	M
FR-M6	Optional collection of real metrics from a controlled test server through an agent or an ingest endpoint	S
FR-M7	Push updates to the UI over WebSocket	S

5.6 Alerts and incidents

ID	Requirement	Prio.
FR-AL1	Configure alert rules: metric (CPU, RAM, availability), operator, threshold, severity (info, warning, critical), target server or global	M
FR-AL2	After each new sample, evaluate active rules and create an alert when a threshold is crossed, without duplicating an already open alert	M
FR-AL3	Alert lifecycle: open, acknowledged, resolved; automatic resolution when the value returns to normal	M
FR-AL4	Incident history with filters (severity, status, server, date range) and pagination	M
FR-AL5	Record who acknowledged or resolved an alert and when	M
FR-AL6	Email or webhook notification on critical alerts	S

5.7 Dashboard

ID	Requirement	Prio.
FR-D1	Counters: projects, servers by status, applications, open alerts by severity	M
FR-D2	Indicators: average CPU and RAM across servers, overall availability	M
FR-D3	Charts: resource usage trend, alerts over time, servers per provider	M
FR-D4	Filters by project, environment and time range	M

ID	Requirement	Prio.
FR-D5	List of the most loaded servers and the latest incidents	S

5.8 Cross-cutting API behaviour

ID	Requirement	Prio.
FR-X1	Every endpoint is documented in Swagger with request and response schemas and bearer auth	M
FR-X2	All inputs are validated by DTOs and ValidationPipe; invalid requests return 400 with field-level messages	M
FR-X3	A global exception filter returns one consistent error format	M
FR-X4	List endpoints support `page`, `limit`, `search`, `sort` and filters, and return total counts	M
FR-X5	An audit log records who created, changed or deleted key resources	S

5.9 Frontend behaviour

ID	Requirement	Prio.
FR-F1	Responsive layout with sidebar navigation (desktop) and collapsible menu (mobile)	M
FR-F2	Login page, route protection, and redirect to login on 401	M
FR-F3	Each list and form screen shows loading, empty and error states	M
FR-F4	UI hides or disables actions the current role is not allowed to do (the backend remains the authority)	M
FR-F5	Forms validate input client-side and display server validation errors	M

5.10 Deployment

ID	Requirement	Prio.
FR-DP1	One Dockerfile each for the backend and the frontend	M
FR-DP2	Docker Compose starts frontend, backend and PostgreSQL with a persistent volume	M
FR-DP3	Configuration through environment variables, with a documented `.env.example`	M
FR-DP4	Database migrations and seed data run automatically or with one documented command	M
FR-DP5	Deployed or demonstrable on a VM, VPS or cloud environment	M

6 Non-functional requirements

Targets are sized for a demo-scale system (tens of users, up to about 100 servers, one metric sample per server per minute) and are measurable on the demo environment.

ID	Category	Requirement	Target / check
NFR-1	Security	Passwords hashed; JWT signed with a secret from the environment, short expiry	No plaintext secret or password in code, DB or logs
NFR-2	Security	Role checks enforced in backend Guards on every protected route	Viewer calling a write endpoint gets 403
NFR-3	Security	Input validation, parameterised queries through the ORM, CORS restricted to the frontend origin, security headers (Helmet), rate limiting on login	No SQL injection on tested inputs; repeated failed logins throttled
NFR-4	Security	Secrets live in environment variables, never committed to Git	`.env` ignored; `.env.example` provided
NFR-5	Performance	Standard API calls respond quickly under normal load	95% of requests under 500 ms with seeded data
NFR-6	Performance	Large lists are paginated; metric history queries use an index on (server_id, timestamp)	Default page size 10 to 20; no unbounded query
NFR-7	Scalability	Backend is stateless (JWT, no server session) so it can be replicated behind a load balancer later	Two backend containers can run side by side
NFR-8	Reliability	Database is persisted in a volume; services restart automatically; a failing job does not crash the API	`restart: unless-stopped`; errors logged
NFR-9	Data integrity	Foreign keys and constraints in the schema; deletes either cascade or are blocked explicitly	No orphan rows after deleting a project or server
NFR-10	Usability	Responsive from 360 px to desktop; consistent navigation; clear loading and error messages	Checked on phone and laptop widths
NFR-11	Maintainability	TypeScript strict mode, ESLint and Prettier, modular NestJS structure (one module per domain), shared naming conventions	Lint passes with no errors
NFR-12	Testability	Jest unit tests on services and guards, plus integration tests on key endpoints (auth, servers, alerts)	Tests run with one command; target about 60% coverage on services
NFR-13	Portability	Whole stack runs from containers on any Docker host	`docker compose up` works on a clean VM
NFR-14	Documentation	Swagger UI, README with install steps, architecture diagram, data model	Available in the repository and kept up to date
NFR-15	Observability	Structured logs and a `/health` endpoint on the backend	Health check used by Compose
NFR-16	Traceability	Feature branches, pull requests and clear commit messages	Every change reaches the main branch through a pull request

7 System architecture

The system is a three-tier application packaged as three Docker containers: a Next.js web app, a NestJS REST API, and a PostgreSQL database.

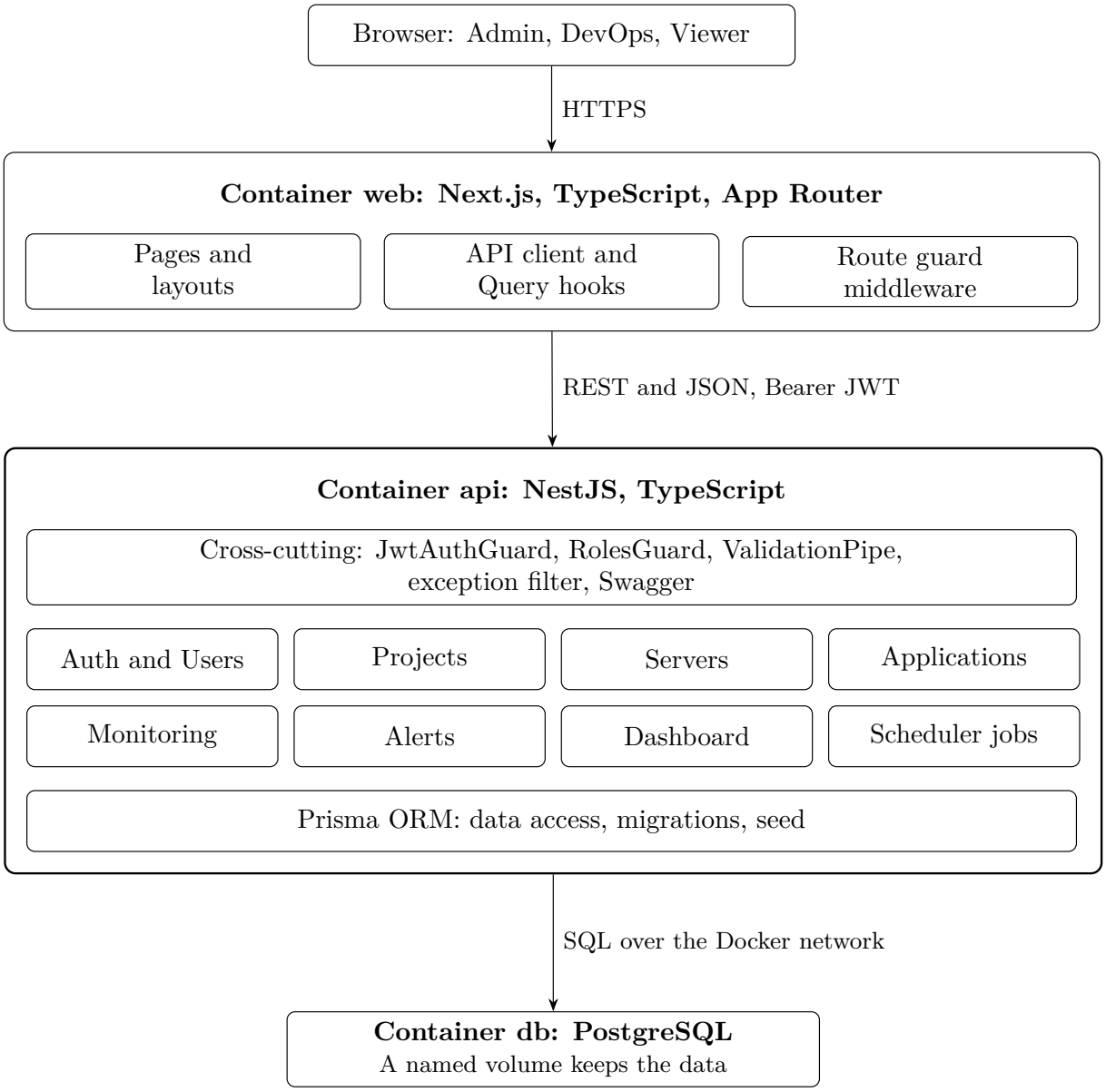


Figure 1: Architecture: only the NestJS API talks to PostgreSQL. Docker Compose starts the three containers; settings come from environment variables.

The browser only reaches the web container. The web app calls the API with a JWT, and only the API reads and writes the database. Guards and pipes run before any module code, so every route is checked for identity, role and input shape.

7.1 Data model

Ten linked entities. All use a UUID or integer primary key plus `createdAt` and `updatedAt`.

Entity	Main fields	Relations
User	email, passwordHash, name, role (ADMIN, DEVOPS, VIEWER), active	Has many ProjectMember, Alert (acknowledged by)
Team	name, description	Has many ProjectMember
Project	name, description, status, startDate, endDate	Has many Environment, Server, ProjectMember
ProjectMember	role in project	Joins User, Team (optional) and Project
Environment	name (dev, staging, production), projectId	Belongs to Project; has many Server and Application
Server	name, provider, region, cpu, ramGb, storageGb, ipAddress, status	Belongs to Project and Environment; has many Application, Metric, AlertRule, Alert
Application	name, version, status	Belongs to Server and Environment
Metric	serverId, cpuPercent, ramPercent, timestamp	Belongs to Server; index on (serverId, timestamp)
AlertRule	metric, operator, threshold, severity, enabled	Belongs to Server (or global)
Alert	severity, status (OPEN, ACKNOWLEDGED, RESOLVED), message, openedAt, resolvedAt	Belongs to Server and AlertRule; links to User who handled it

7.2 API design and conventions

Module	Base route	Main endpoints
Auth	`/auth`	POST login, POST refresh, GET me
Users	`/users`	GET list, GET one, POST, PATCH, DELETE (Admin only)
Projects	`/projects`	CRUD, plus members and environments sub-routes
Servers	`/servers`	CRUD, GET `/:id/metrics`, GET `/:id/availability`
Applications	`/applications`	CRUD
Monitoring	`/metrics`	POST ingest sample, GET history
Alert rules	`/alert-rules`	CRUD
Alerts	`/alerts`	GET list, PATCH acknowledge, PATCH resolve
Dashboard	`/dashboard`	GET summary, GET trends

This gives roughly 35 endpoints in total.

- Resources are plural nouns; JSON in and out; standard status codes (200, 201, 400, 401, 403, 404, 409).
- Lists accept `page`, `limit`, `search`, `sort` and resource filters, and return `data` plus `meta` (total, page, limit).
- One error shape: `statusCode`, `message`, `error`, `path`, `timestamp`.
- Swagger UI is served at `/api/docs` with a Bearer auth button.

7.3 Code organisation

- **Backend:** one NestJS module per domain (controller, service, DTOs, Prisma access), plus `common/` for guards, decorators, filters and pipes.
- **Frontend:** App Router with route groups for `(auth)` and `(dashboard)`, a `lib/api` client, reusable components, and one folder per feature.
- **Repository:** a single Git repository with `backend/`, `frontend/`, `docker-compose.yml`, `.env.example` and `docs/`.

7.4 Docker Compose setup

Docker Compose is the single way to run the whole stack, in development and in the demo. One `docker compose up -build` starts the database, the API and the web app in the right order.

Service	Image / build	Port	Key settings
`db`	`postgres:16-alpine`	5432 (internal only in production)	Named volume `pgdata`; credentials from `.env`; healthcheck with `pg_isready`
`api`	Build from `./backend` (multi-stage Dockerfile)	3001	Waits for `db` to be healthy; runs `prisma migrate deploy` then starts; `DATABASE_URL`, `JWT_SECRET`, `JWT_EXPIRES_IN`; healthcheck on `/health`
`web`	Build from `./frontend` (multi-stage Dockerfile)	3000	Waits for `api`; `NEXT_PUBLIC_API_URL` points to the API

Rules.

- All services share one Compose network and reach each other by service name (for example `db:5432`).
- Only `web` and the API (for Swagger) are published to the host; the database is not exposed in the demo environment.
- Every service uses `restart: unless-stopped`, and `depends_on` with `condition: service_healthy` controls start order.
- Configuration lives in `.env` (ignored by Git), documented by `.env.example`. No secret is written in the Compose file or the images.
- Dockerfiles are multi-stage (build, then a slim runtime image) and run as a non-root user.

Files.

- `docker-compose.yml`: base definition, used for the demo and the VM.
- `docker-compose.override.yml`: development extras such as source bind mounts, hot reload, and an optional database admin tool. Compose merges it automatically on a developer machine.
- `.env.example`: variable names with safe sample values.

Daily commands. `docker compose up -build` starts everything, `docker compose logs -f api` follows the API logs, `docker compose exec api npx prisma migrate dev` runs a migration, and `docker compose down` stops the stack while keeping the data (`down -v` also erases the volume).

The same Compose file is what gets copied to a VM or cloud instance later, so the move to AWS or Azure needs no change to the application.

8 Technology choices

The table gives the choice made for each layer and the reason for it. Cloud is deliberately postponed: the same containers run on a VM now and on AWS or Azure later.

Layer	Choice	Why
Frontend framework	Next.js (App Router) with TypeScript	File-based routing, layouts, server and client components, middleware for route protection
Backend framework	NestJS with TypeScript	Modules, dependency injection, Guards, Pipes and filters match the security and validation requirements
API style and docs	REST with `@nestjs/swagger`	Swagger UI is generated from controllers and DTOs
Authentication	`@nestjs/jwt`, Passport JWT strategy, bcrypt (or argon2)	Standard JWT flow; roles through a custom `@Roles()` decorator and `RolesGuard`
Scheduling	`@nestjs/schedule`	Cron-style jobs for metric simulation and threshold evaluation, no extra infrastructure
Database	PostgreSQL 16	Relational model with foreign keys fits the linked entities; good time-series queries with indexes
ORM	Prisma (default choice) or TypeORM	Prisma gives a readable schema, typed queries and simple migrations; pick one at the start and keep it
Containers	Docker and Docker Compose	Three services: `web`, `api`, `db`, with a named volume for PostgreSQL
T Source control	Git and GitHub with feature branches and pull requests	Reviewable history and a shared workflow
Real-time (extension)	NestJS WebSocket gateway (Socket.IO)	Replaces polling for live charts

Cloud later. Nothing in the application depends on a specific provider. Environment variables carry all configuration. When deployment moves to the cloud, the likely path is the same Compose stack on a VM (EC2 or Azure VM), then optionally a managed database (RDS or Azure Database for PostgreSQL) and a container registry (ECR or ACR). This stays a later decision and is not required for the first release.

9 Constraints, assumptions and delivery phases

Constraints.

- Fixed stack: Next.js, NestJS, TypeScript, PostgreSQL, Docker, Jest.
- Real metric collection only on a controlled test infrastructure.
- The whole system must start from containers with no manual installation steps beyond Docker.

Assumptions.

- Monitoring data is simulated by default; real collection is an optional extension.

- One user has one role; there is a single organisation (no multi-tenancy).
- The user interface is in English only.
- Network access to the demo environment is available during demonstrations.

Out of scope.

- Provisioning or controlling real cloud resources (starting, stopping or billing servers).
- Multi-tenant organisations, SSO and OAuth providers.
- Native mobile apps.
- Cost management and billing reports.

Delivery phases.

Phase	Focus	Output
1	Analysis and foundations	Requirements, architecture, Git repository, project skeletons
2	Data and API structure	Entities, migrations, NestJS modules
3	Security and API quality	JWT, Guards, roles, validation, Swagger
4	Web interface	Next.js app, authentication flow, API integration
5	Inventory modules	Projects, servers and applications complete
6	Supervision	Monitoring, charts, alerts, dashboard
7	Packaging and quality	Docker Compose, tests, deployment on a VM
8	Hardening and documentation	Fixes, README, technical documentation